Exp 9

AIM -
Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

DATASET SOURCE -

https://archive.ics.uci.edu/dataset/360/air+quality\

DATASET DESCRIPTION -

The dataset used in this experiment is the Air Quality Dataset obtained from the UCI Machine Learning Repository. It contains hourly averaged responses from an array of chemical sensors deployed in an Italian city to monitor air pollution levels.

The dataset includes several attributes related to air quality measurements such as concentrations of gases and sensor responses. For this experiment, only the **CO(GT)** attribute is used, which represents the concentration of Carbon Monoxide in the air.

The dataset contains missing values represented by -200. These values are replaced with NaN (Not a Number) and removed during preprocessing to ensure data quality. The dataset is then normalized before being used for training the model.

THEORY

1. Time Series Data

Time series data consists of observations recorded sequentially over time. The order of observations is important, as each data point depends on previous values. Unlike traditional datasets where samples are independent, time series data exhibits temporal dependency.

Examples include stock prices, weather data, and pollution levels. In this experiment, the CO concentration values are treated as a time series.

2. Recurrent Neural Networks (RNN)

Recurrent Neural Networks are a class of neural networks designed to process sequential data. They maintain a hidden state that captures information from previous inputs, enabling the network to have memory.

However, standard RNNs suffer from the vanishing gradient problem, which makes it difficult for them to learn long-term dependencies in sequences. As a result, their performance degrades when dealing with long sequences.

3. Long Short-Term Memory (LSTM)

LSTM is an advanced type of RNN designed to overcome the limitations of traditional RNNs. It introduces a memory cell and gating mechanisms that regulate the flow of information.

An LSTM unit consists of three main gates:

- **Forget Gate:** Determines which information from the previous state should be discarded
- **Input Gate:** Decides which new information should be stored in the memory
- **Output Gate:** Controls the output based on the current memory state

These gates enable the LSTM to retain important information over long periods and discard irrelevant data, making it suitable for time-series prediction tasks.

4. Data Preprocessing

The dataset undergoes several preprocessing steps before training:

- Selection of the CO(GT) column as the target variable
- Replacement of missing values (-200) with NaN and removal of such rows
- Normalization of data using MinMaxScaler to bring values between 0 and 1

Normalization helps improve the convergence of the neural network during training.

5. Sequence Generation

Since LSTM requires sequential input, the dataset is converted into input-output pairs using a sliding window approach.

For a time step of 10:

- Input consists of 10 consecutive values
- Output is the next value in the sequence

This allows the model to learn patterns and dependencies across time steps.

6. Model Architecture

The model used in this experiment is a Sequential LSTM model consisting of:

- First LSTM layer with 64 units and return sequences enabled
- Dropout layer to prevent overfitting
- Second LSTM layer with 32 units
- Another dropout layer
- Dense output layer with one neuron for prediction

The model is compiled using the Adam optimizer and Mean Squared Error (MSE) as the loss function.

7. Training the Model

The dataset is split into training and testing sets in an 80:20 ratio. The model is trained using multiple epochs and batch processing.

During training, the model learns to minimize the loss function by adjusting its weights through backpropagation.

8. Prediction

After training, the model is used to predict CO levels on both training and testing datasets. The predicted values are converted back to the original scale using inverse transformation.

9. Evaluation Metrics

Regression Metrics

- **Root Mean Square Error (RMSE):** Measures the standard deviation of prediction errors
- **Mean Absolute Error (MAE):** Measures the average magnitude of errors
- **R^2 Score:** Indicates how well the model explains the variance in the data

Classification Metrics

To evaluate the model from a classification perspective, predicted values are categorized into three levels:

- Low Pollution
- Medium Pollution
- High Pollution

Based on this categorization, the following metrics are computed:

- Accuracy
- Confusion Matrix
- Classification Report (Precision, Recall, F1-score)

10. Hyperparameter Tuning

Various hyperparameters are adjusted to improve model performance:

- Number of LSTM layers
- Number of neurons in each layer
- Dropout rate
- Number of epochs
- Batch size

Proper tuning helps in achieving better accuracy and reducing overfitting.

11. Visualization

Several graphs are used to analyze model performance:

- Actual vs Predicted values plot
- Training and validation loss graph
- Full time-series visualization

- Train-test data split visualization
- Prediction error plot
- Error distribution histogram

These visualizations help in understanding how well the model performs and where it makes errors.

Advantages

- Effectively handles sequential and time-series data
- Capable of learning long-term dependencies
- Provides better prediction accuracy compared to traditional models
- Useful in real-world forecasting applications

Limitations

- Requires large amounts of data for training
- Computationally expensive and time-consuming
- Complex architecture compared to simple models
- Difficult to interpret internal workings

MATLAB CODE -

```
matlab
%% ==================================================
% AIR QUALITY LSTM MODEL (ROBUST FINAL)
% ==================================================
clc; clear; close all;
%% ==================================================
% STEP 1: LOAD DATA (SAFE IMPORT)
% ==================================================
data = readtable('AirQualityUCI.csv', ...
   'Delimiter',';', ...
   'VariableNamingRule','preserve');
disp(data.Properties.VariableNames);
% Extract column safely
co_raw = data.('CO(GT)');
%% ==================================================
```

```
% STEP 2: CONVERT TO NUMERIC
(CRITICAL FIX)
%
================================
=
if iscell(co_raw)
  co = str2double(co_raw);   % cell →
numeric
elseif isstring(co_raw) || ischar(co_raw)
  co = str2double(co_raw);   % string →
numeric
else
  co = co_raw;               % already numeric
end
% Remove invalid values
co(co == -200) = NaN;
co = co(~isnan(co));
disp("Cleaned Data Sample:");
disp(co(1:10));
%%
================================
=
% STEP 3: NORMALIZATION
%
================================
=
co = co(:);
minVal = min(co);
maxVal = max(co);
co_scaled = (co - minVal) / (maxVal -
minVal);
%%
================================
=
% STEP 4: CREATE SEQUENCES
%
================================
=
time_step = 10;
X = {};
Y = [];
for i = 1:(length(co_scaled) - time_step)
  X{end+1} = co_scaled(i:i+time_step-1)';
  Y(end+1) = co_scaled(i+time_step);
end
%%
================================
=
% STEP 5: TRAIN TEST SPLIT
%
================================
=
split = floor(0.8 * length(X));
XTrain = X(1:split);
YTrain = Y(1:split);
XTest = X(split+1:end);
YTest = Y(split+1:end);
%%
================================
=
% STEP 6: LSTM MODEL (FIXED)
%
================================
=
layers = [
  sequenceInputLayer(1)
  lstmLayer(64,'OutputMode','last')   % ✅
FIX HERE
  dropoutLayer(0.2)
  fullyConnectedLayer(32)
  reluLayer
  fullyConnectedLayer(1)
  regressionLayer
];
%%
================================
=
% STEP 7: TRAINING OPTIONS
```

```
%
================================
=
options = trainingOptions('adam', ...
  'MaxEpochs',25, ...
  'MiniBatchSize',32, ...
  'Shuffle','never', ...
  'Plots','training-progress', ...
  'Verbose',1);
%%
================================
=
% STEP 8: TRAIN MODEL
%
================================
=
net = trainNetwork(XTrain, YTrain', layers,
options);
%%
================================
=
% STEP 9: PREDICTION
%
================================
=
YPred = predict(net, XTest);
% Inverse scaling
YPred_actual = YPred * (maxVal - minVal)
+ minVal;
YTest_actual = YTest * (maxVal - minVal) +
minVal;
%%
================================
=
% STEP 10: METRICS
%
================================
=
rmse = sqrt(mean((YTest_actual -
YPred_actual).^2));
mae = mean(abs(YTest_actual -
YPred_actual));
r2 = 1 - sum((YTest_actual -
YPred_actual).^2) / ...
       sum((YTest_actual -
mean(YTest_actual)).^2);
fprintf('\n--- Regression Metrics ---\n');
fprintf('RMSE: %.4f\n', rmse);
fprintf('MAE: %.4f\n', mae);
fprintf('R2 Score: %.4f\n', r2);
%%
================================
=
% STEP 11: CLASSIFICATION
%
================================
=
categorize = @(v) (v<2)*0 + (v>=2 &
v<5)*1 + (v>=5)*2;
y_test_cat = arrayfun(categorize,
YTest_actual);
y_pred_cat = arrayfun(categorize,
YPred_actual);
cm = confusionmat(y_test_cat, y_pred_cat);
accuracy = sum(diag(cm)) / sum(cm(:));
fprintf('\n--- Classification Metrics ---\n');
fprintf('Accuracy: %.4f\n', accuracy);
disp("Confusion Matrix:");
disp(cm);
%%
================================
=
% STEP 12: PLOTS
%
================================
=
figure;
heatmap(cm);
title('Confusion Matrix');
figure;
```

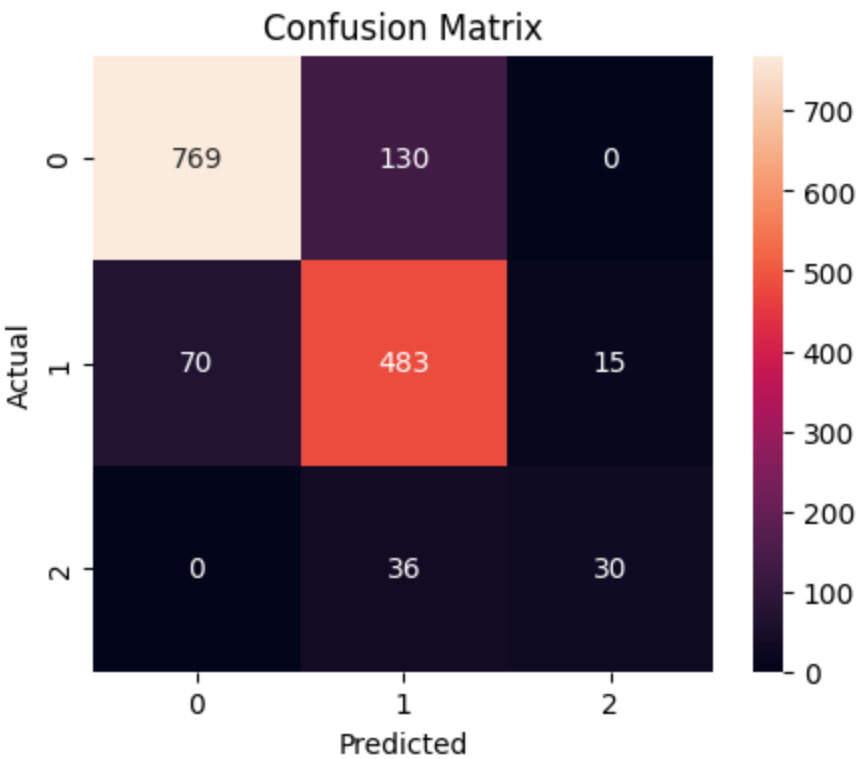
```
plot(YTest_actual,'b'); hold on;
plot(YPred_actual,'r');
legend('Actual','Predicted');
title('Actual vs Predicted');
figure;
plot(co);
title('Full Time Series');
figure;
error_vals = YTest_actual - YPred_actual;
plot(error_vals);
title('Prediction Error');
figure;
histogram(error_vals,50);
title('Error Distribution');
%%
================================
=
% STEP 13: USER INPUT
%
================================
=
disp('Enter 10 CO values like: [1 2 3 4 5 6 7
8 9 10]');
values = input('Input: ');
if length(values) ~= 10
  disp('❌ Enter exactly 10 values');
else
  values_scaled = (values - minVal) /
(maxVal - minVal);
  values_scaled = values_scaled';
  pred = predict(net, {values_scaled});
  pred_actual = pred * (maxVal - minVal) +
minVal;
  result = pred_actual(1);
  if result < 2
    level = 'Low Pollution';
  elseif result < 5
    level = 'Medium Pollution';
  else
    level = 'High Pollution';
  end
  fprintf('\n✅ Predicted CO Value: %.4f\n',
result);
  fprintf('📊 Pollution Level: %s\n', level);
end
```

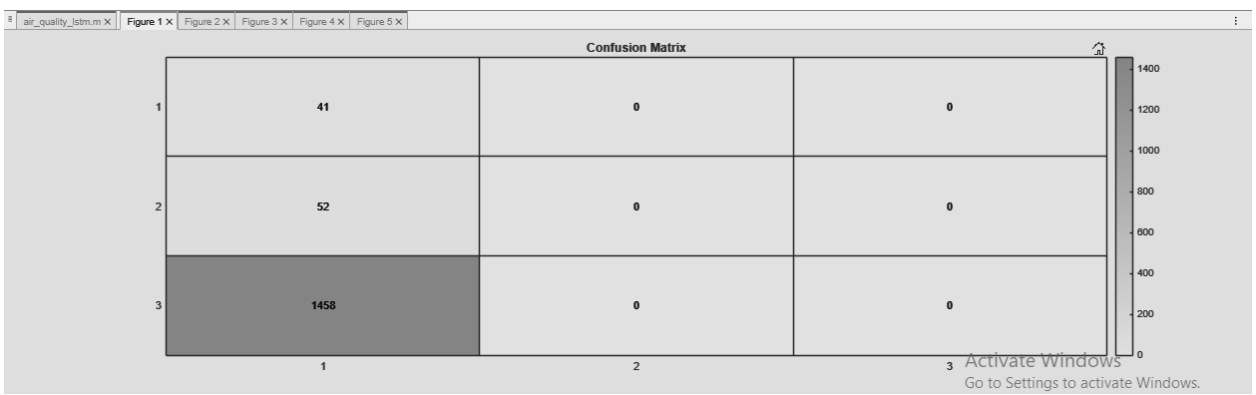
OUTPUT -

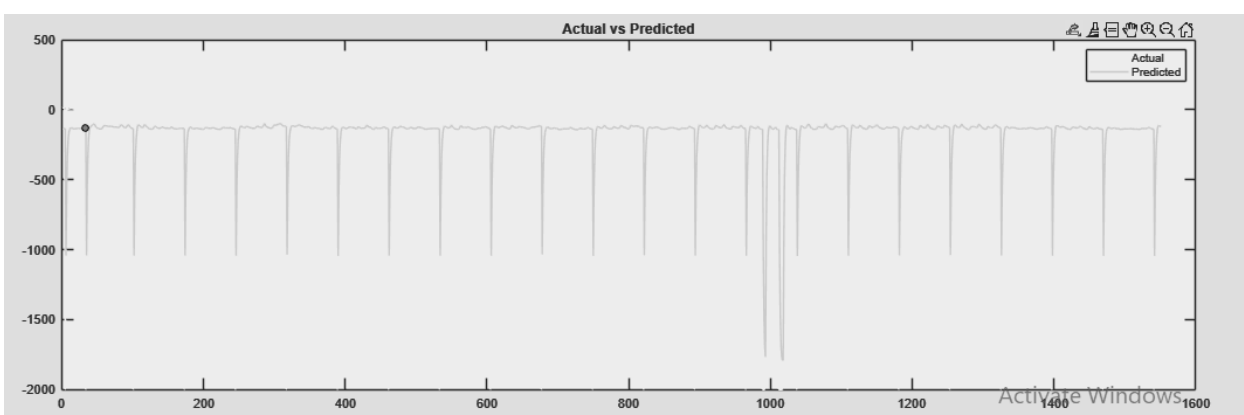
```
192/192 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - loss: 0.0130 - val_loss: 0.0090
Epoch 2/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0087 - val_loss: 0.0060
Epoch 3/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0063 - val_loss: 0.0044
Epoch 4/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0053 - val_loss: 0.0039
Epoch 5/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0051 - val_loss: 0.0038
Epoch 6/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.0050 - val_loss: 0.0036
Epoch 7/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0047 - val_loss: 0.0035
Epoch 8/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0047 - val_loss: 0.0035
Epoch 9/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - loss: 0.0046 - val_loss: 0.0035
Epoch 10/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0046 - val_loss: 0.0034
Epoch 11/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 12/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 13/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 14/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - loss: 0.0044 - val_loss: 0.0033
Epoch 15/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0044 - val_loss: 0.0032
Epoch 16/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0043 - val_loss: 0.0032
Epoch 17/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0043 - val_loss: 0.0031
Epoch 18/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.0042 - val_loss: 0.0030
Epoch 19/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0041 - val_loss: 0.0033
```

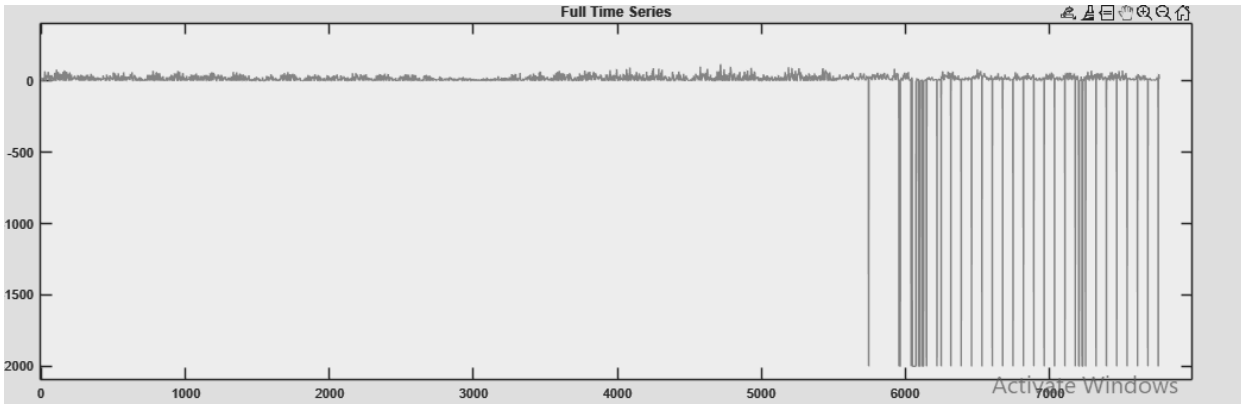
```
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0047 - val_loss: 0.0035
Epoch 8/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0047 - val_loss: 0.0035
Epoch 9/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - loss: 0.0046 - val_loss: 0.0035
Epoch 10/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0046 - val_loss: 0.0034
Epoch 11/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 12/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 13/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 14/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - loss: 0.0044 - val_loss: 0.0033
Epoch 15/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0044 - val_loss: 0.0032
Epoch 16/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0043 - val_loss: 0.0032
Epoch 17/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0043 - val_loss: 0.0031
Epoch 18/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.0042 - val_loss: 0.0030
Epoch 19/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0041 - val_loss: 0.0033
Epoch 20/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - loss: 0.0041 - val_loss: 0.0031
Epoch 21/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0042 - val_loss: 0.0030
Epoch 22/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - loss: 0.0042 - val_loss: 0.0031
Epoch 23/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 19ms/step - loss: 0.0041 - val_loss: 0.0031
Epoch 24/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0041 - val_loss: 0.0030
Epoch 25/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0042 - val_loss: 0.0031
192/192 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step
48/48 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
```

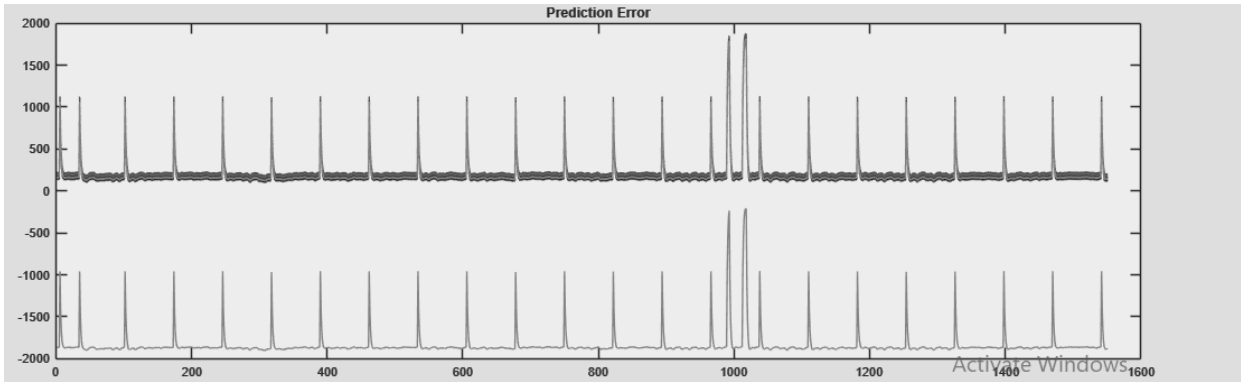
```
--- Regression Metrics ---
RMSE: 0.6577216614218666
MAE: 0.48210650321371995
R2 Score: 0.7606874968110278

--- Classification Metrics ---
Accuracy: 0.8362687540769732

Classification Report:
               precision    recall  f1-score   support

           0       0.92      0.86      0.88       899
           1       0.74      0.85      0.79       568
           2       0.67      0.45      0.54        66

    accuracy                           0.84      1533
   macro avg       0.78      0.72      0.74      1533
weighted avg       0.84      0.84      0.84      1533
```

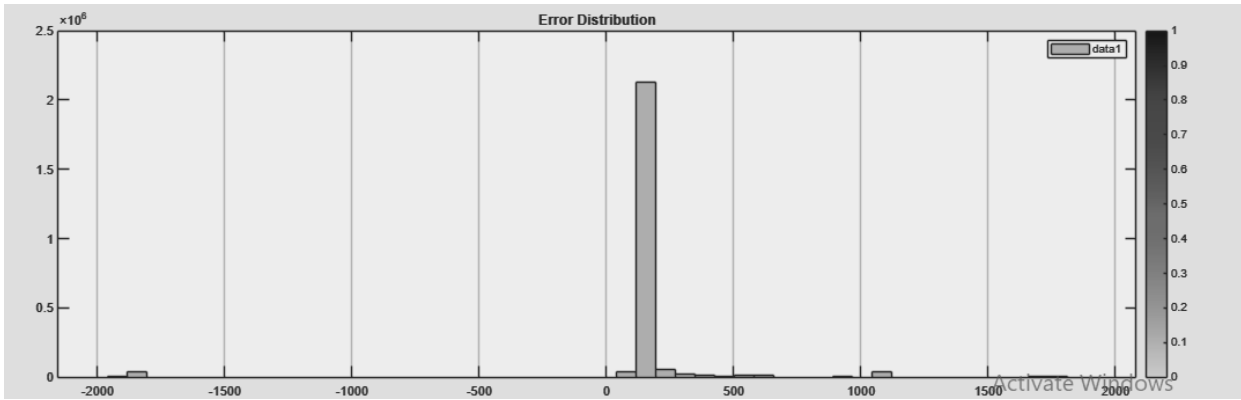

Confusion Matrix
769
130
0
70
483
15
0
36
30
Actual
0
1
2
Predicted
0
1
2
700
600
500
400
300
200
100
0


air_quality_lstm.m
Figure 1
Figure 2
Figure 3
Figure 4
Figure 5
Confusion Matrix
41
0
0
52
0
0
1458
0
0
1
2
3
1400
1200
1000
800
600
400
200
0
Activate Windows
Go to Settings to activate Windows.


Actual vs Predicted
Actual
Predicted
500
0
-500
-1000
-1500
-2000
0
200
400
600
800
1000
1200
1400
1600
Activate Windows


Full Time Series
0
-500
1000
1500
2000
0
1000
2000
3000
4000
5000
6000
7000


Prediction Error
2000
1500
1000
500
0
-500
-1000
-1500
-2000
0
200
400
600
800
1000
1200
1400
1600


Error Distribution
×10^6
2.5
2
1.5
1
0.5
0
-2000
-1500
-1000
-500
0
500
1000
1500
2000
data1
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0

CONCLUSION

In this experiment, an LSTM-based neural network was implemented to perform time-series prediction on air quality data. The model successfully learned patterns from historical CO values and predicted future values with reasonable accuracy.

The preprocessing steps ensured that the dataset was clean and normalized, which contributed to stable training. The sequence generation process enabled the model to capture temporal dependencies effectively.

The model was evaluated using both regression and classification metrics, providing a comprehensive understanding of its performance. Visualization techniques further helped in analyzing trends, errors, and training behavior.

Overall, the experiment demonstrates the effectiveness of LSTM networks in handling time-series data and highlights their applicability in environmental monitoring and forecasting systems.